

From “Wow It Works!” to “Wow It Still Works”

Making vibecoded apps production-worthy in five steps

Andrés García · Senior Software Engineer

2026-07-23

Opening · The Hook

Already deployed!

Button changes color → "it's ready." That gap is everything.

```
# "Guys, I've already built my first website using AI..."
$ deploy --app prototype # ✓ ready for production
# → http://localhost:8000
```

Vibecoding — Andrej Karpathy (Feb 2025). Word of the Year 2025 — Collins · Merriam-Webster.

Source: [Karpathy, A. — X post, Feb 2025](#) · Collins Dictionary, Word of the Year 2025

The Real-World Pattern

Two anecdotes, same lesson

"We put it together in two hours with AI, we just need to deploy it." — Analytics Lead

The 5-day pitch was a visual marvel. Then:

- No exception handling
- DB coupled to views
- Collapsed >10 concurrent users

We confused local success with architectural resilience.



The Problem • The Real Speed

Play vs. work — and the uncomfortable data

AI-ASSISTED DEVELOPMENT

Supervised pair-programmer

Review code, migrations, deps before accepting.

PURE VIBE CODING

Acceptance without review

“Click to see if it works” — full-stack in hours, zero lines written.

+24%

Perceived speed

-19%

Real speed (METR)

AMDAHL’S LAW

Bottleneck = reviewing, not generating.

Sources: [METR \(2025\)](#) · [Uplevel \(2024\)](#)

The Programmer's New Role

From Author to Curator

80%

of **Anthropic**'s production code written by Claude.

1/3

of historical outage bugs caught by **automated CI/CD reviewer** before production.

Source: [Anthropic Institute \(May 2026\)](#)

No longer authors of syntax. We are curators, auditors, validators.

SURVIVAL MANUAL • PART 1

The Diagnosis of Chaos

The Physics of Synthetic Chaos

01

Diagnosis · The Three-Month Wall

Predictable degradation + invisible debt

MONTHS 1–3 · EUPHORIA

Extraordinary speed

Full-speed feature releases.
Perceived velocity: peak.

MONTHS 4–9 · PLATEAU

Integration friction

Redundant code paths.
Debugging AI code consumes
all cycles.

THEN · COLLAPSE

Development halts

Team can't maintain or explain
its own codebase.

4x–8x

more code duplication

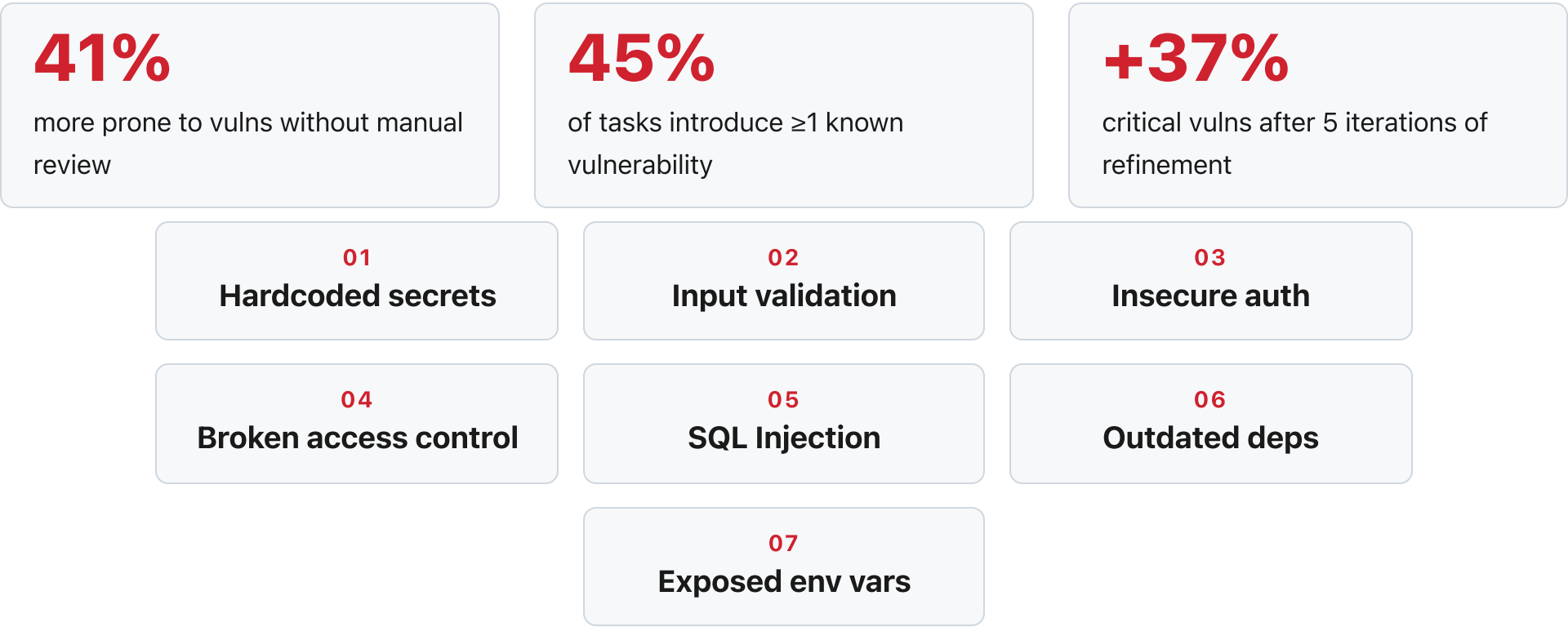
25% →
<10%

refactoring fell (2021 → 2024)

Sources: [GitClear \(2024\)](#)

Diagnosis · The Technical Heart

Where synthetic code fails exactly



Sources: [Perry et al. \(2023\)](#) · [Tihanyi et al. \(2024\)](#) · [Shukla et al. \(2025\)](#)

Diagnosis · The Risk No One Sees

Slopsquatting and Token Exfiltration

DEPENDENCY HALLUCINATION

FRONTIER MODELS

5.2%

invents packages

OPEN-SOURCE
MODELS

21.7%

invents packages

Attackers register malicious packages at hallucinated names. Merge without verifying → **arbitrary code in production.**

Sources: [Lanyado et al. \(2024\)](#) · [Socket Security \(2025\)](#) · [WEF \(2025\)](#)

NX CASE · AUGUST 2025

Exfiltrated tokens from **Claude Code, Gemini CLI, Amazon Q.**

By 2027 → cybersecurity = skill #1 for engineers.



Your turn

Have you hit “the three-month wall”? What broke first — the code, the team’s understanding, or both?

SURVIVAL MANUAL • PART 2

The Solution

Five Steps to Govern Synthetic Code

02

Manual · Step 0 · Threat Model

Step 0

Hardening without a threat model = checklist security. 10 min with **STRIDE** on your data flow:

Spoofing

Who impersonates a user/service?

Tampering

Data modified in transit?

Repudiation

Actions denied without audit?

Info Disclosure

Leaks through logs/errors?

Denial of Service

Unbounded ops exhaust resources?

Elevation

User reaches admin functions?

Owner: Tech Lead + Security Champion · **Time:** 10–15 min/feature · **Output:** Threat map prioritizing Steps 1–5

Manual · Step 1 · Sanitation and Secrets

Step 1

01 · PURGE



TruffleHog

`git filter-repo` / BFG.

Rotate all credentials +
configure automated
rotation (OIDC, short-lived
tokens).

02 · ISOLATE



Secret Manager

Secret Manager (Vault / AWS
/ GCP). Inject via workload
identity — no long-lived
keys.

03 · SCAN



Container Registry

Gate: image with **critical**
CVEs → blocked from
deploy.

Manual · Step 2 · Stateless Infrastructure

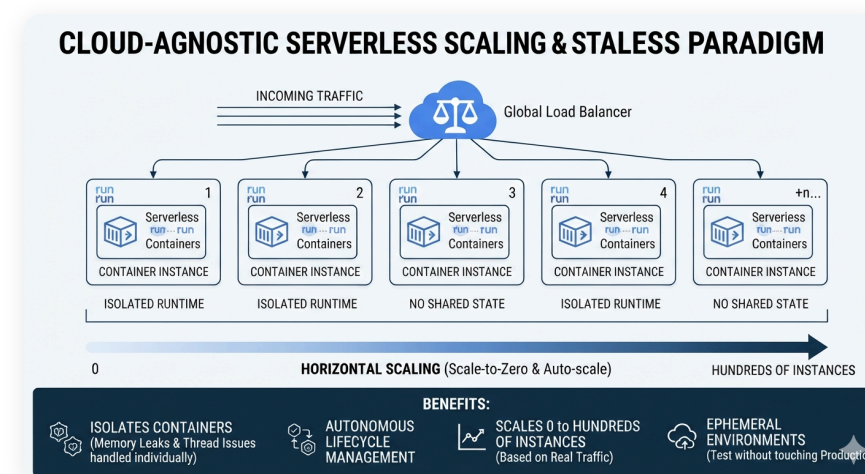
Step 2

DB coupled to dev server = destroyed scaling.

Serverless containers (Cloud Run · Fargate · Container Apps) enforce stateless paradigm.

- Isolates memory leaks and bad threads
- Scales 0 → hundreds on real traffic
- Concurrency limits as **cost gates** — prevents AI unbounded loops from exponential bills
- Ephemeral envs — test without touching prod

IaC (Terraform/Pulumi/CDK) — no manual drift. 12-factor compliance enforced.



Manual · Step 3 · Data Hardening

Step 3

01

Connection Pooling

Reuse channels with **explicit pool size limits**. No per-request sockets.

02

Cursor-Based Pagination

Keyspace pagination at scale. Strict `limit` on every read. No RAM DoS.

03 · KEY

Row-Level Security

DB-enforced isolation. Kills IDOR / broken access control.

04

Encryption at Rest

CMEK for sensitive data. Classify tiers: PII, financial, public.

Manual · Step 4 · Unified QA

Step 4

Isolated unit tests aren't enough. Under **real concurrency**, experience degrades.

- **K6** — Stress ramp against services + DB in the cloud.
- **Playwright** — Real users, real browsers, simultaneously.
- **OpenTelemetry** — Structured logs + distributed tracing + metrics.



Correlate rendering failures with **infrastructure saturation**. Define **SLOs** (p95 < 200ms, error < 0.1%) — error budgets trigger review.

Manual · Step 5 · Perimeter Governance

Step 5



Web Security Scanner

dynamic pentest → ephemeral Cloud
Run URLs

DAST scanner (OWASP ZAP · Nuclei · Web Security Scanner) on live ephemeral URLs.

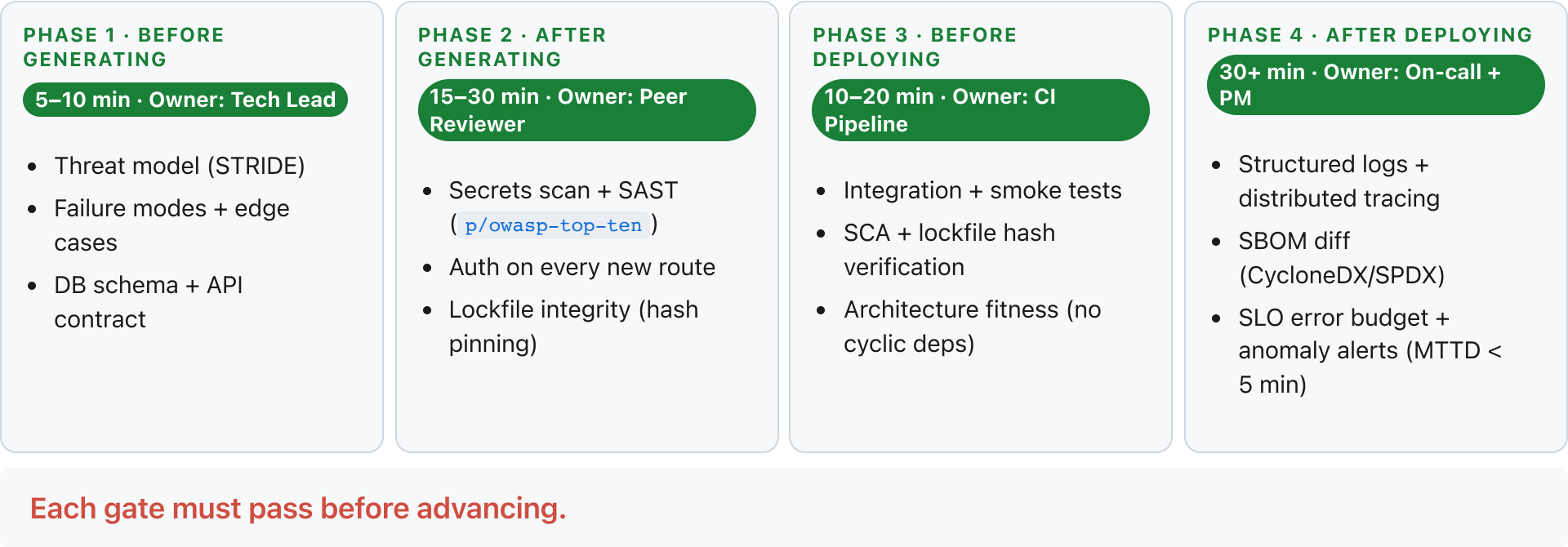
Detects: - Runtime injection + XSS - Misconfigured headers (CSP, HSTS, X-Frame-Options) - Dev server signature leaks

Policy deviation → **blocks container promotion.**

Runtime: WAF + rate limiting in prod. mTLS between services.
Egress filtering for AI-generated code.

Manual · Actionable Checklist · DevSecOps

Gate Verification · DevSecOps



SURVIVAL MANUAL · PART 3

The Methodology Behind the Manual

Spec-Driven Development (SDD)

Separate **what** (business) from **how** (syntax) — specs as non-negotiable contracts.

03

SDD · Architecture + Enforcement

Specs as contracts, hooks as barriers

01

Constitution

Purpose, boundaries, decisions

mission.md

tech-stack.md

boundaries.md

02

Specifications

One folder per feature

requirements.md

plan.md

validation.md

03

P-I-V Loop

Plan → Implement → Validate

agent reviews plan

spec as system prompt

tests vs contracts

BRAIN 🧠

Reasons, debugs, recovers. Specs in
Markdown = no token cost for policy memory.

MUSCLES · 0 TOKENS

Shell hooks as hard barriers: [check-specs.sh](#)
· [check-tests.sh](#) · [check-api.sh](#)

Supported by **Kiro (AWS)** · **Spec Kit (GitHub)** · **Tessl**. The agent cannot skip them.

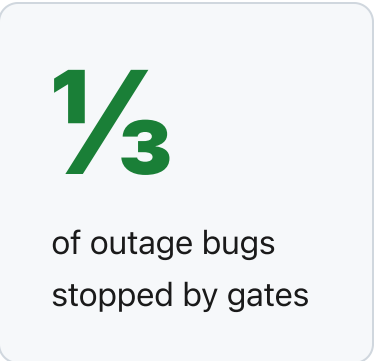
SDD · Why It Works

Traceability + Impact

GIT HOOK · PREPARE-COMMIT-MSG

```
commit a3f9c1 · feat: conversational form

Generated-by: Claude-Code-Agent
Reviewed-by: Lead Architect
Spec-ref: specs/features/auth
Audit-ID: AUD-2026-0411
```



Gate automation = the **only containment barrier** that scales with synthetic code generation.

SDD · Maturity Model

Where Are You Today?



Start Level 1 — reach Level 2 in one sprint. Each level unlocks the next.

Key Takeaways

Remember These 5 Things

1 · VIBECODING ≠ SHIPPING

Demo ≠ product. Local success ≠ resilience.

2 · REVIEW > GENERATE

Bottleneck moved. You're a curator now.

3 · GATES ARE NON-NEGOTIABLE

Secrets · stateless · data · QA · perimeter.
Skip one, pay later.

4 · SPECS BEFORE PROMPTS

Contracts in Markdown first. Agent implements against them.

5 · AUTOMATE DISCIPLINE

Hooks + CI gates = barriers agents can't skip. Zero tokens.

6 · Fund the investment — 5 steps ≈ 1 sprint. A breach ≈ [\\$4.4M \(IBM, 2025\)](#). Budget it or accept the risk explicitly.

Connect. Learn. Grow.

Build the future with AI speed and the **unbreakable rigor** of true engineering.

↗ goo.gle/vibecoding-manual



Questions?

Your anecdotes dealing with AI code, and how to implement these gates in your workflows. The videocam is yours.

Sources & Credits

Original content: Carlos Prieto · Google Developer Expert · GDG Bogotá
Talk “¿Y ahora qué?” — Build with AI 2026

Productivity & Trends

- [Karpathy — “Vibecoding” \(2025\)](#)
- [Collins — Word of the Year 2025](#)
- [METR — AI Dev Productivity \(2025\)](#)
- [Uplevel — AI Productivity \(2024\)](#)
- [Anthropic Institute — “When AI Builds Itself” \(2026\)](#)
- [GitClear — “Coding on Copilot” \(2024\)](#)

Security & Risks

- [Perry et al. — AI Code Security \(2023\)](#)
- [Tihanyi et al. — AI-Generated Code \(2024\)](#)
- [Shukla et al. — Iterative AI Security \(2025\)](#)
- [Lanyado et al. — Package Hallucinations \(2024\)](#)
- [Socket — Nx S1ngularity Attack \(2025\)](#)
- [WEF — Future of Jobs 2025](#)
- [IBM — Cost of a Data Breach 2025](#)